```
      # nothing
    end

    LOOP_COUNT.times do |x|
      method
    end
  end
end
```

produces:

```
Rehearsal ------------------------------------------------
inline:        0.020912   0.000031   0.020943 (  0.020943)
method:        0.030763   0.000179   0.030942 (  0.030941)
--------------------------------------- total: 0.051885sec

                   user     system      total        real
inline:        0.021142   0.000036   0.021178 (  0.021184)
method:        0.030943   0.000036   0.030979 (  0.030986)
```

The bm or bmbm methods take a block and execute the block while calculating the time that the block takes, and reporting that result in four columns:

- The time in seconds used by the CPU executing the user process.
- The time in seconds used by the CPU in system calls during the bock.
- The sum of the first two columns.
- The actual amount of elapsed time during the block, sometimes called "clock time" or "wall time."

Because the resulting value is in seconds, lower values are faster, which we only mention because some timing tools give a "executions per second" measure, and that can be confusing.

The argument to bm or bmbm is the width of the columns of the output.

You may be wondering what the difference is between bm and bmbm. It has to do with trying to make the output of the benchmarking test more consistent.

Ruby programs can run slowly because of the overhead of garbage collection. Because this garbage collection can happen any time during your program's execution, you may find that benchmarking gives misleading results, showing a section of code running slowly when in fact the slowdown was caused because garbage collection happened to trigger while that code was executing. The bmbm method, therefore, runs the tests twice—once as a rehearsal and once to actually measure performance—in an attempt to minimize the distortion introduced by garbage collection. The official docs, we note, say "there's only so much that bmbm can do, and the results are not guaranteed to be isolated from garbage collection and other effects."

The benchmarking process itself is relatively well mannered—it won't slow down your program much.

What's Next

In this chapter, we talked about different ways to debug Ruby, including printing information to the console, using the official Ruby debugger, and using the popular third-party program, Pry. We also looked at how to use Ruby's benchmarking tools to identify slow spots in your code.